



Câu lạc bộ An toàn thông tin
ĐH BKHN

Return by Death

A Re:Zero-Themed CTF Challenge



Presenter 1: Haiyahhh
@Haiyahhh



Agenda

- 1 Concept
- 2 Background You Need
- 3 Challenge Architecture
- 4 The Kill Chain
- 5 Design Rationale
- 6 Demo

1 Concept

Theme & Core Lesson

- Re:Zero: die, reset, try again
- Four vulnerabilities, one chain
 - ▶ DOM Clobbering
 - ▶ TOCTOU race condition
 - ▶ Kubernetes Pod lifecycle
 - ▶ Insecure deserialization (PyYAML)
- White/Gray-box: source & manifests provided

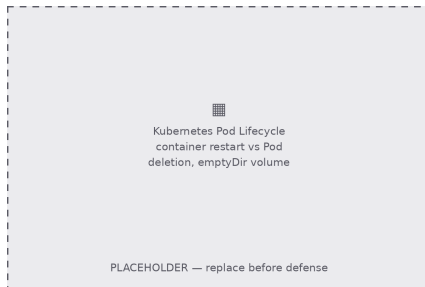


2 Background You Need

Kubernetes 101

Pods, volumes, policies

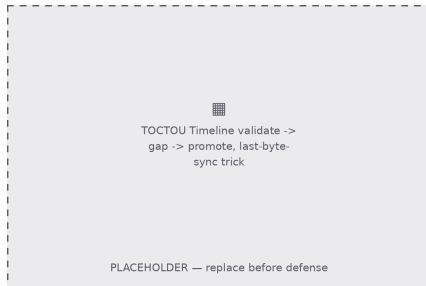
- Container crash $\neq$ Pod deletion
- `emptyDir` volume survives a container restart
- `readOnlyRootFilesystem`: blocks webshells
- `NetworkPolicy`: pod-level firewall



TOCTOU & Last-Byte-Sync Race

Time-of-check to time-of-use

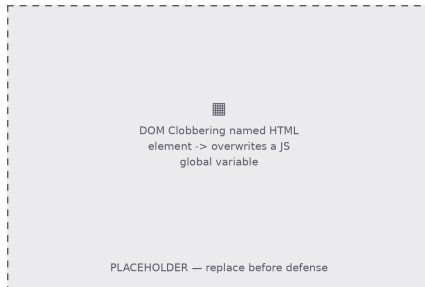
- Validate a value, then act on a *different* read of it
- Classic gap: check file, re-read file, act
- Last-byte-sync: hold two requests one byte short
- Release both bytes together → handlers race in lockstep



DOM Clobbering

HTML that overwrites JavaScript

- Named HTML elements become global JS variables
- `id="x"` can clobber `window.x` if `x` is undefined
- No `<script>` tag needed — pure markup
- Sanitizers that allow forms/ids don't stop this
- Trusted script's own config lookup gets redirected



Insecure Deserialization

When data becomes code

- Serialize: object $\rightarrow$ bytes. Deserialize: bytes $\rightarrow$ object
- Some formats let bytes pick *which class* to build
- PyYAML's `yaml.Loader`: builds arbitrary Python objects
- `!!python/object/apply` tag $\rightarrow$ calls a function directly
- `yaml.safe_load`: plain data types only, no tags



3 Challenge Architecture

Two-Tier Deployment

- Web Pod (Flask) + DB Pod (Postgres)
- Kind cluster, fully portable
- Flag lives only inside Postgres





Defenses In Place

- `readOnlyRootFilesystem: true`
- `NetworkPolicy`: DB ingress from Web Pod only
- `emptyDir`: `/app/cache`, `/tmp`
- Goal: block the trivial shells and bypasses



4 The Kill Chain

Overview



Kill Chain Overview Stage ->

Race -> Death -> Return

PLACEHOLDER — replace before defense



Stage 1 — Payload Staging

- Register a standard account
- Upload malicious `profile.yml` as a "backup"
- Lands in `profile.staging.yml` only



Stage 2 — The Race

- `/health` validates the staged file (`safe_load`)
- Promotion re-reads the file from disk
- Race: swap in the payload after validation, before promotion



Stage 3 — The Death

- Bio clobbers `window.STEWARD_STATUS_CONFIG`
- Support ticket → admin bot visits profile
- Clobbered fetch hits `/maintenance/restart`
- Pod calls `os._exit(1)`



Stage 4 — The Return

- Kubernetes restarts the container; `emptyDir` survives
- Boot sequence deserializes `profile.yml` with `yaml.Loader`
- `DatabaseExporter` gadget fires
- SQL query travels the allowed `NetworkPolicy` path
- Flag POSTed to an external webhook

5 Design Rationale



Anti-Slop Design

- Fake metrics / logs / node-status endpoints
- Scanners chase SSRF and RCE that isn't there
- `safe_load` blocks the "obvious" upload
- Forces understanding, not guessing

Reliability



- Race outcome is probabilistic, not deterministic
- Solve script retries in bursts
- Waits for Pod recovery between attempts
- Designed to converge eventually, every time

6 Demo

Live Demo



Live demo screen recording /
screenshot

PLACEHOLDER — replace before defense



Thank you!



Presenter 1: Haiyahhh
@Haiyahhh